

U.S. Labor Market Analytics Pipeline

An End-to-End Cloud Data Engineering Portfolio Project

Azure Data Factory • PostgreSQL • Tableau

Project Report

Prepared by Swamini Deepak Sontakke

Data Window: 2016 – 2025

Table of Contents

Table of Contents.....	2
1. Motivation and Objective	3
1.1 Motivation	3
1.2 Objective.....	3
2. Data Sources	3
3. Database Architecture.....	4
3.1 Design Approach	4
3.2 Dimension Tables	4
3.3 Fact Tables.....	4
3.4 Entity Relationship Diagram	4
4. Azure Infrastructure.....	5
5. ADF Pipeline Architecture	5
5.1 Ingestion Pipelines.....	6
5.2 Transformation Pipelines and Data Flows	6
6. Results: Dashboards and Business Questions Answered.....	7
6.1 Dashboard 1-Labor Market Health	7
6.2 Dashboard 2-Sector Employment & Wages	8
6.3 Dashboard 3-Occupation Wage Analysis	9
7. Data Validation and Quality Checks	10
7.1 Row-Count Validation.....	10
7.2 Referential Completeness	10
7.3 Outlier Investigation: Confirming a Real Result vs. a Bug	10
7.4 Calculated-Field Cross-Checks.....	10
8. Challenges and How They Were Resolved	11
8.1 BLS API Key Timing	11
8.2 Column Name Conflict Between Fact and Dimension Metadata	11
8.3 Blob Storage Soft Delete Blocking Data Flow Reads	11
8.4 Silent Date-Type Mismatches in Lookups.....	11
8.5 The OES Pipeline: Multi-Generational Schema Drift	11
8.6 Decimal Type Consistency Through the Transformation Chain	12
9. Limitations.....	12
10. Future Enhancements.....	13
11. Conclusion.....	13

1. Motivation and Objective

1.1 Motivation

Labor market data in the United States is published across multiple federal sources-the Federal Reserve Economic Data (FRED) system, and several surveys run by the Bureau of Labor Statistics (BLS)-each with its own access method, refresh cadence, and data shape. Analysts who want a unified view of labor market health, sector-level employment and wage trends, and occupation-level wage analysis typically have to manually download, clean, and reconcile these sources before any meaningful analysis can begin.

This project was built to demonstrate the ability to design and implement a production-style, cloud-native data pipeline that automates this entire process-from raw API/file ingestion through to a query-ready analytical database and a set of interactive business intelligence dashboards. It serves as a portfolio piece to showcase practical, hands-on proficiency with the Azure data engineering stack, relational data modeling, and BI visualization, using a real-world dataset rather than a synthetic or toy example.

1.2 Objective

The objective was to design and build a complete, end-to-end analytics pipeline that:

- Ingests labor market data from three distinct public data sources (FRED, BLS CES, BLS OES) using Azure Data Factory.
- Transforms and conforms that data into a clean, well-modeled star schema hosted on a managed PostgreSQL database.
- Presents the resulting data through a set of interactive Tableau dashboards addressing distinct, real business/economic questions.
- Follows sound data engineering practice throughout-secure credential handling, modular pipeline design, idempotent loads, and defensive handling of inconsistent source data.

The finished pipeline covers ten years of monthly and annual labor market data (2016–2025) and is fully reproducible: every stage, from raw ingestion to final dashboard, runs through versioned, parameterized ADF pipelines rather than one-off manual scripts.

2. Data Sources

Three public data sources were used, each capturing a different dimension of the U.S. labor market:

Source	Description	Granularity	Coverage
FRED JOLTS	Job Openings and Labor Turnover Survey-job openings, hires, total separations, quits, and layoffs/discharges, pulled via the St. Louis Fed API (5 series: JTSJOL, JTSHIL, JTSTSL, JTSQUL, JTSLDL).	Monthly, national	2016–2025
BLS CES	Current Employment Statistics-employment, average hourly earnings, and average weekly hours across 9 major supersectors (28 series total).	Monthly, by sector	2016–2025
BLS OES	Occupational Employment Statistics-total employment, mean hourly wage, and mean annual wage across 23 major occupation groups, national cross-industry estimates.	Annual, by occupation	2016–2025

FRED JOLTS and BLS CES data were retrieved programmatically through their respective public APIs. BLS OES data, by contrast, is distributed only as downloadable Excel workbooks (no public API for this specific cut

of the data), and was manually downloaded and staged into Blob Storage for ingestion-a distinction that turned out to matter a great deal during development, discussed further in Section 6.

3. Database Architecture

3.1 Design Approach

The analytical layer is modeled as a star schema in PostgreSQL (schema: analytics), chosen specifically because it optimizes for the read-heavy, aggregate-oriented query patterns that BI tools like Tableau generate, keeps dimension logic (dates, sectors, occupations, metrics) centralized and reusable across fact tables, and keeps the fact tables themselves narrow and fast to scan.

3.2 Dimension Tables

Table	Grain	Key Columns
dim_date	One row per calendar month (120 rows) plus one row per year for annual data (10 rows)	date_key, full_date, year, month, month_name, quarter, is_annual
dim_sector	One row per BLS supersector (10 rows, incl. Total Nonfarm)	sector_key, sector_name
dim_occupation	One row per BLS OES major occupation group (23 rows)	occ_key, occ_code, occ_title, occ_group, occ_category
dim_metric	One row per distinct measured metric across all sources (11 rows)	metric_key, metric_code, metric_name, metric_category, unit_of_measure, source

3.3 Fact Tables

Table	Grain	Row Count	Key Columns
fact_labor_market	One row per date × sector (or national) × metric, covering both JOLTS and CES data	3,720 (JOLTS: 600, CES: 3,120)	date_key, sector_key, metric_key, value, source, load_timestamp
fact_occupation_wages	One row per year × occupation × metric	690 (23 occ × 3 metrics × 10 yrs)	date_key, occ_key, metric_key, value, yoy_change, yoy_pct_change, source, load_timestamp

Both fact tables use a composite natural key (date_key, sector_key/occ_key, metric_key) enforced through the sink's upsert configuration, making every pipeline run idempotent-re-running a load for a given period updates existing rows rather than creating duplicates.

3.4 Entity Relationship Diagram

The diagram below shows the complete star schema. Both fact tables share the dim_date and dim_metric dimensions-the same date and metric reference data is reused across the monthly JOLTS/CES data and the annual OES data-while each fact table also joins to its own dedicated dimension (dim_sector for labor market data, dim_occupation for wage data).

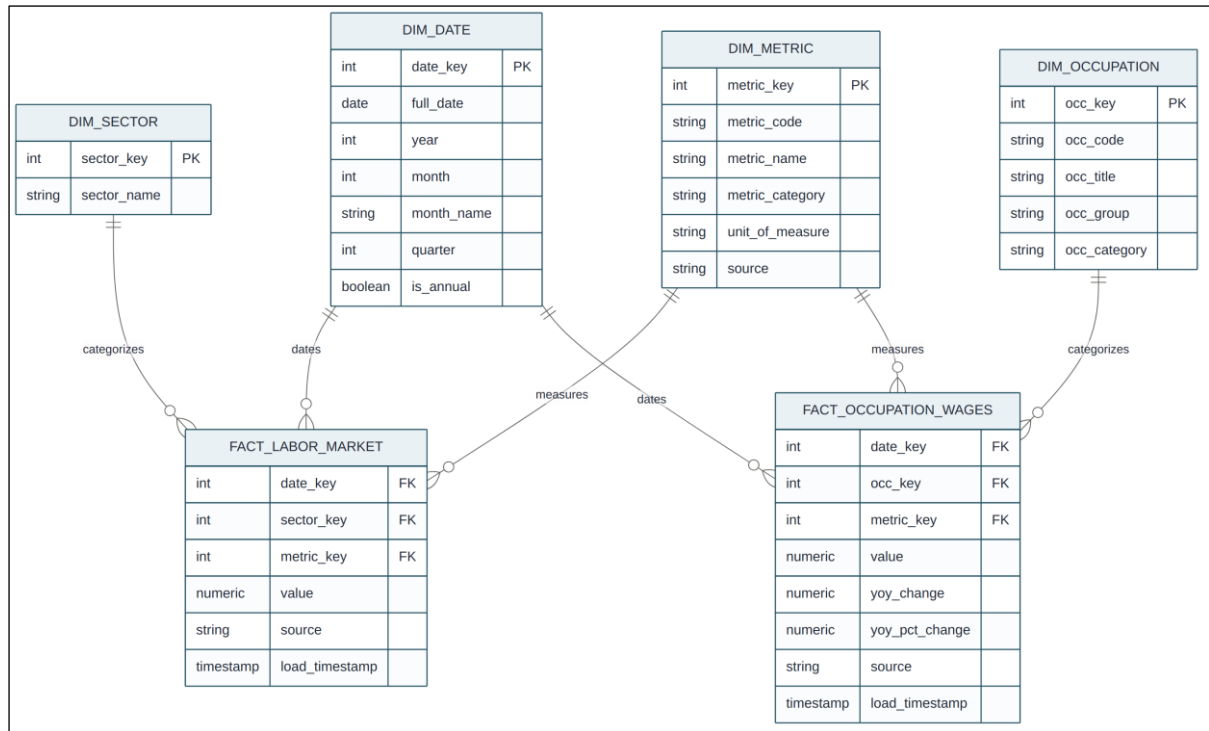


Figure 1. Star schema ER diagram-fact_labor_market and fact_occupation_wages with shared and dedicated dimensions.

4. Azure Infrastructure

All resources are grouped under a single resource group for lifecycle management and cost tracking:

Resource	Name	Purpose
Resource Group	Rg-Labour-Analytics	Logical container for all project resources
Storage Account	stlabouranalytics	Blob storage for raw ingested files (JOLTS, CES, OES)
Key Vault	kv-labour-analytics	Secure storage of API keys and database credentials
PostgreSQL Flexible Server	pg-labour-analytics	v15, Burstable B1ms tier-the analytical data warehouse (DB: labor_analytics, schema: analytics)
Data Factory	(pipeline orchestrator)	Hosts all ingestion pipelines and data flows

Credentials for both the BLS API and the PostgreSQL server are stored in Key Vault and referenced by Linked Services at runtime-no credentials are hardcoded anywhere in pipeline or data flow definitions. A Burstable B1ms PostgreSQL tier was chosen deliberately, since this is a portfolio-scale workload (low, bursty query volume) rather than a production system needing sustained high throughput.

5. ADF Pipeline Architecture

The pipeline layer is split into two independent stages-ingestion and transformation-rather than one monolithic pipeline. Each stage is triggered and run separately, which keeps re-runs cheap (a failed transform can be re-run without re-pulling source data) and keeps each pipeline's blast radius small.

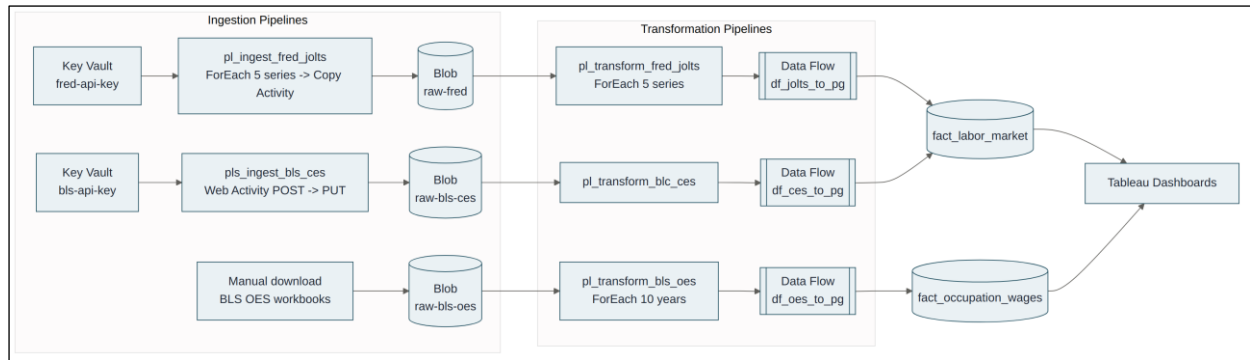


Figure 2. End-to-end pipeline architecture-ingestion pipelines landing raw data in Blob Storage, transformation pipelines executing the Data Flows into PostgreSQL, feeding Tableau.

5.1 Ingestion Pipelines

Pipeline	Pattern	Notes
pl_ingest_fred_jolts	Web Activity (GET key from Key Vault) → ForEach over 5 JOLTS series IDs → Copy Activity per series	Lands raw JSON at raw-fred/jolts/ in Blob Storage
pls_ingest_bls_ces	Web Activity (GET key from Key Vault) → Web Activity (POST to the BLS API, 28 series in one request) → Web Activity (PUT response to Blob Storage)	Lands one combined JSON file at raw-bls-ces/ces/ per run, parameterized by start_year/end_year
(OES ingestion)	Manual download of Excel workbooks, uploaded directly to Blob Storage	No public API exists for this specific national cross-industry cut of OES data, so this stage is manual by necessity; everything downstream is fully automated

5.2 Transformation Pipelines and Data Flows

Three transformation pipelines each wrap a single Data Flow behind an Execute Data Flow activity, parameterized so the same Data Flow logic runs once per series/year:

Pipeline	Orchestration	Data Flow	Sink
pl_transform_fred_jolts	ForEach over 5 series → Execute Data Flow per iteration	df_jolts_to_pg	fact_labor_market
pl_transform_blc_ces	Single Execute Data Flow activity (all 28 series processed in one Data Flow run)	df_ces_to_pg	fact_labor_market
pl_transform_bls_oes	ForEach over 10 years (2016–2025) → Execute Data Flow per iteration	df_oes_to_pg	fact_occupation_wages

The JOLTS and CES Data Flows follow a common structural pattern, despite pulling from different API response shapes:

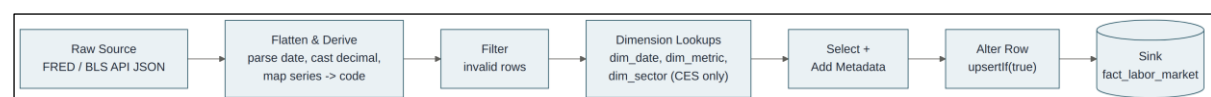


Figure 3. Shared Data Flow transformation pattern used by df_jolts_to_pg and df_ces_to_pg.

Two design rules, learned through direct debugging experience, are applied consistently across all three flows:

- The Alter Row transformation must be the very last transformation before the Sink.
- Metadata columns (source, load_timestamp) must be added in one final Derived Column placed immediately before the Sink-adding them earlier in the chain causes them to be silently dropped or to collide with identically named columns pulled in through a dimension lookup.

df_oes_to_pg follows the same overall shape but required substantially more transformation logic to handle inconsistent source files and a platform limitation in the native Unpivot transformation-detailed in full in Section 7.5 and Section 7.6:

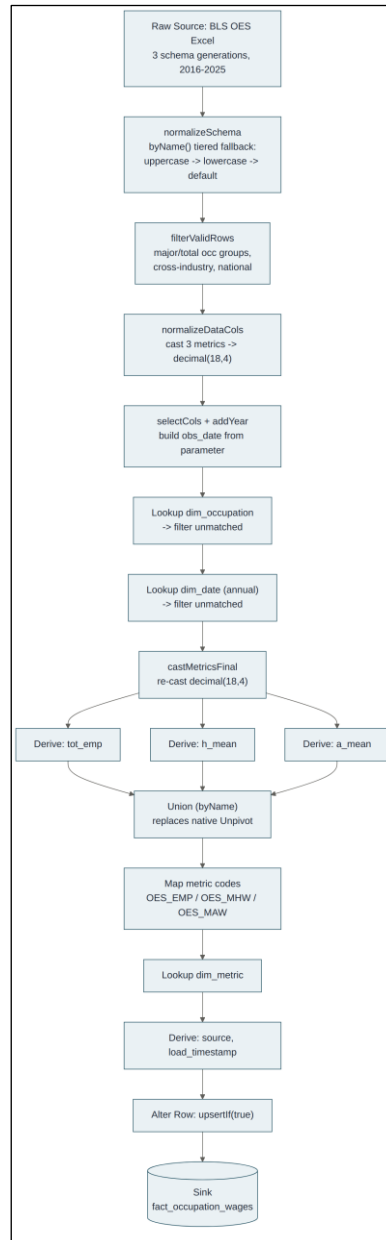


Figure 4. df_oes_to_pg in full-schema normalization tier, dimension lookups, and the Union-based reshape that replaced the native Unpivot transformation.

6. Results: Dashboards and Business Questions Answered

The three resulting Tableau dashboards each answer a distinct set of practical, real-world labor market questions:

6.1 Dashboard 1-Labor Market Health

Built on FRED JOLTS data, this dashboard addresses questions such as:

- Is the labor market tightening or loosening over time-are there more job openings than hires, and how has that gap evolved?
- How did the labor market behave through COVID-19 and its recovery, compared to pre-pandemic norms?
- Are workers quitting voluntarily at a higher rate than they are being laid off-a leading indicator of worker confidence?

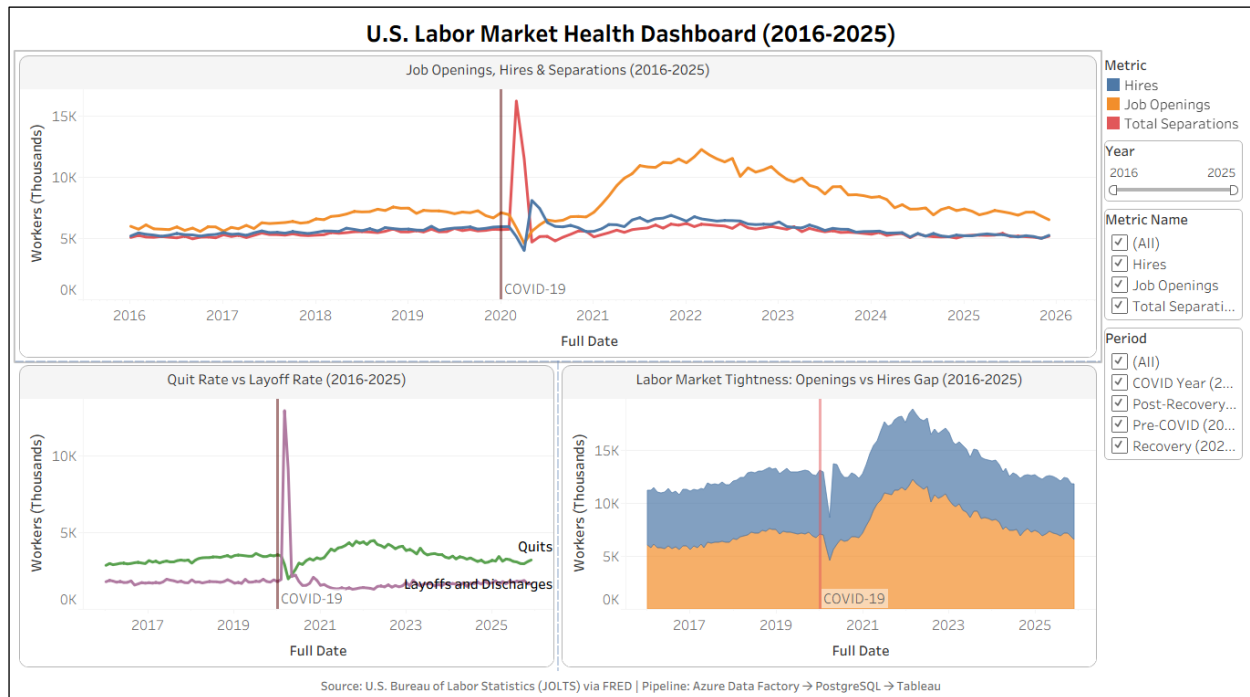


Figure 5. Dashboard 1-Labor Market Health.

6.2 Dashboard 2-Sector Employment & Wages

Built on BLS CES data, this dashboard addresses:

- Which sectors are growing or shrinking in employment share over the past decade?
- Which sectors pay the highest average hourly wages, and how does this compare across industries?
- Which sectors have the longest or shortest average work weeks-a proxy for labor intensity and potential overtime cost exposure?

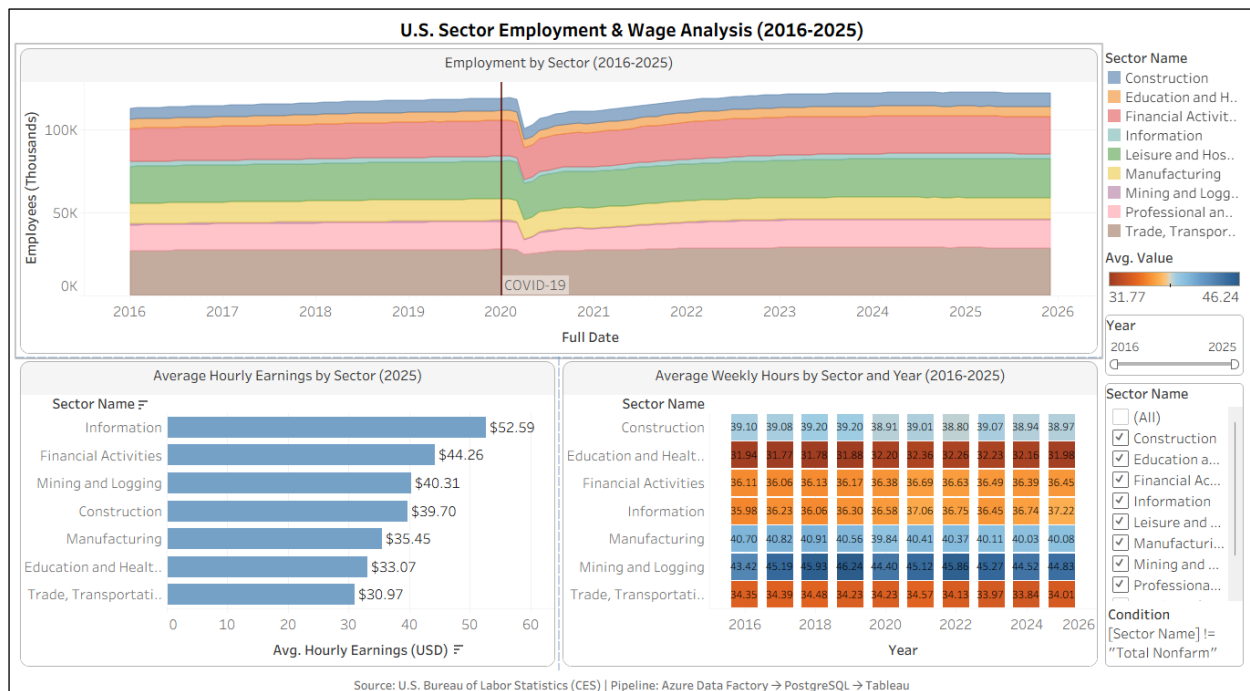


Figure 6. Dashboard 2-Sector Employment & Wages.

6.3 Dashboard 3-Occupation Wage Analysis

Built on BLS OES data, this dashboard addresses:

- Which occupations command the highest pay nationally, and how does that compare to their overall workforce size?
- Where is there a trade-off between high pay and low employment share (e.g., legal, specialized science occupations) versus high employment share and moderate pay (e.g., office and administrative support)?
- Which occupations have seen the fastest wage growth from 2016 to 2025-useful for identifying which fields are becoming relatively more competitive for talent, or where wage inflation has been most pronounced?

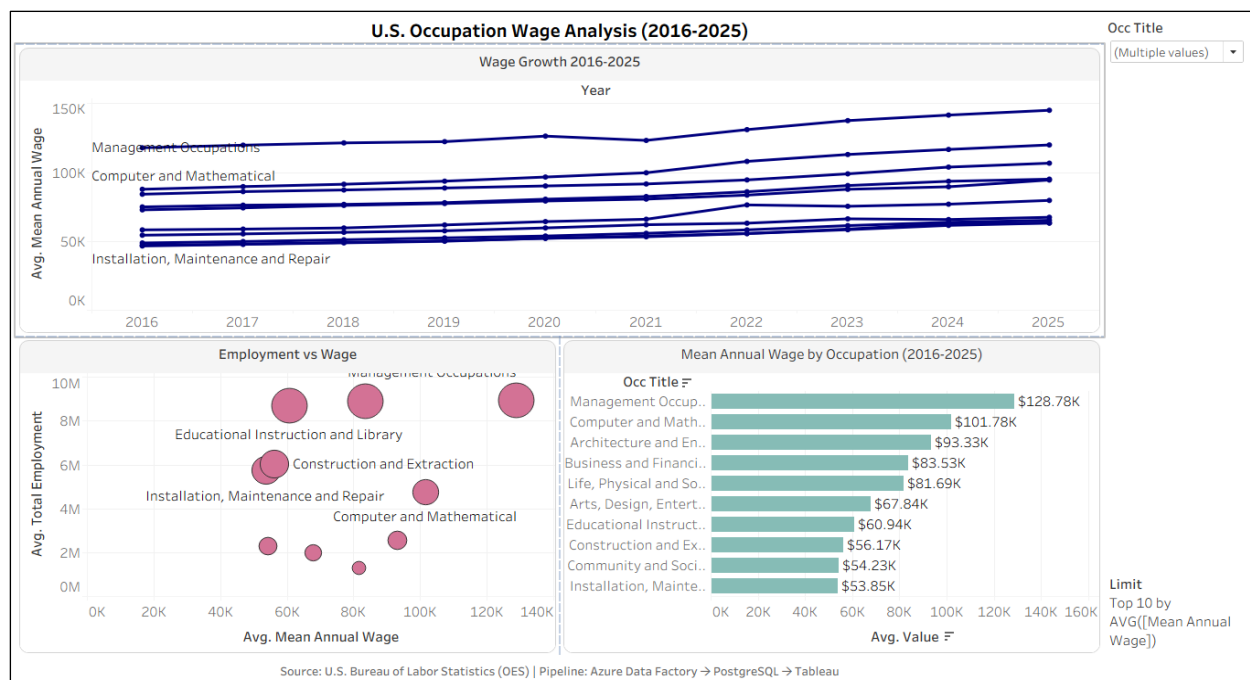


Figure 7. Dashboard 3-Occupation Wage Analysis.

Collectively, these three views support use cases across several audiences: workforce development agencies assessing where to focus training investment, job seekers and career changers evaluating pay and growth trends by field, HR and compensation teams benchmarking against sector or occupation-level wage trends, and economic researchers or policy analysts tracking labor market tightness and its relationship to wage growth.

7. Data Validation and Quality Checks

Throughout development, each pipeline stage was validated against expected row counts and cross-checked with manual calculations before being trusted as a foundation for the Tableau layer. This section documents the checks that actually run against the finished data, both as evidence of correctness and as a template for validating future loads.

7.1 Row-Count Validation

After each backfill, row counts were verified directly in PostgreSQL against the expected shape of each fact table:

Fact Table	Expected Shape	Verification Query Result
fact_labor_market (FRED_JOLTS)	5 series $\times$ 120 months = 600 rows	600 rows confirmed
fact_labor_market (BLS_CES)	26 usable series $\times$ 120 months = 3,120 rows	3,120 rows confirmed
fact_occupation_wages (BLS_OES)	23 occupations $\times$ 3 metrics $\times$ 10 years = 690 rows	690 rows confirmed, 69 rows/year across all 10 years (2016–2025)

7.2 Referential Completeness

Beyond raw row counts, each dimension join was checked for completeness rather than just assuming a non-zero row count meant success:

- Metric-level balance: `SELECT metric_code, COUNT (*)` grouped by metric confirmed exactly 230 rows per metric (OES_EMP, OES_MHW, OES_MAW)-an even split proving lookupMetric resolved cleanly for every row rather than silently dropping one metric type.
- Occupation coverage: `SELECT COUNT (DISTINCT occ_key) FROM fact_occupation_wages` confirmed all 23 major occupation groups from dim_occupation were represented, ruling out a partial-match issue in lookupOcc.
- Year coverage: a year-by-year `GROUP BY` confirmed all 10 years (2016–2025) were present with identical row counts, catching the schema-drift bug described in Section 8.5 before it could reach the dashboard layer.

7.3 Outlier Investigation: Confirming a Real Result vs. a Bug

During construction of the Employment vs. Wage scatter plot (Dashboard 3), the “Farming, Fishing, and Forestry” occupation group appeared as a near-zero-employment outlier, visually inconsistent with the rest of the dataset. Rather than assuming this was a pipeline defect, the underlying value was investigated directly before being accepted into the dashboard.

The result was confirmed to be a legitimate feature of the source data rather than an error: BLS OES estimates only wage-and-salary employment, and most agricultural workers are self-employed or work on family-run farms-a population OES structurally excludes. This explains why this occupation group's OES-reported employment is far smaller than its true real-world workforce size. This distinction is called out directly in the dashboard's supporting documentation so viewers don't mistake it for a data quality issue.

7.4 Calculated-Field Cross-Checks

Tableau calculated fields (e.g., Wage Growth %, built as a Level of Detail expression comparing 2016 and 2025) were spot-checked against manual arithmetic on raw values pulled directly from the database for a sample of

occupations before being trusted across the full dataset—for example, confirming Healthcare Support's reported 33.90% growth against the raw wage values (30,470 → 40,800) by hand. This practice caught an earlier, incorrect version of the same calculation that had blended three different metrics together (Section 8.6 discusses the related type-handling issue that first surfaced this class of bug).

8. Challenges and How They Were Resolved

This project surfaced a number of non-obvious platform behaviors in Azure Data Factory. Documenting them here both as a record of problem-solving and as a reference for future pipeline work.

8.1 BLS API Key Timing

The CES ingestion pipeline (`pls_ingest_bls_ces`) calls a Web Activity to retrieve an API key from Key Vault, then needs that key inside the body of a subsequent Web Activity that calls the BLS API. Initially, the key reference was structured in a way that could resolve before the retrieval activity had actually completed, leaving the key empty and causing repeated 403 Forbidden errors from the BLS API.

- **Resolution:** reference the key retrieval activity's output directly inside the dependent activity's expression (`activity('act_get_bls_api_key').output.value`), with an explicit `dependsOn/Succeeded` condition between the two activities. Because ADF only evaluates an activity's expressions after its declared dependencies have succeeded, this guarantees the key is fully resolved before it is used, without needing a separate Set Variable activity in between.

8.2 Column Name Conflict Between Fact and Dimension Metadata

The CES pipeline began failing on a NOT NULL constraint violation for a column named `source`. The fact table's own metadata column `source` (tracking data provenance) collided with an identically named column carried through from a `dim_metric` lookup.

- **Resolution:** restructure the flow so that the metadata columns (`source`, `load_timestamp`) are populated in one dedicated Derived Column placed immediately before the Sink, eliminating the naming collision entirely.

8.3 Blob Storage Soft Delete Blocking Data Flow Reads

Data Flows reading directly from Blob Storage intermittently failed until soft delete was identified as the cause—the feature interferes with how Data Flows enumerate and read blob contents.

- **Resolution:** disable soft delete at the storage account level (`az storage account blob-service-properties update --enable-delete-retention false`).

8.4 Silent Date-Type Mismatches in Lookups

Joining ADF Data Flow date columns against PostgreSQL date columns that failed silently in places—rows simply didn't match, with no error thrown, because of subtle timezone/precision differences between how Spark (the Data Flow execution engine) and PostgreSQL represent dates internally.

- **Resolution:** always cast both sides of a date-based lookup condition to an explicit string format (`toString(date_col, 'yyyy-MM-dd')`) before comparing, rather than comparing native date types directly applied in `df_ces_to_pg`'s `addDateString/dimDataStr` transformations.

8.5 The OES Pipeline: Multi-Generational Schema Drift

This was the most significant engineering challenge in the project. The BLS OES data is distributed as separate annual Excel workbooks, and across the 2016–2025 window, BLS quietly changed the file's internal column layout twice, producing three distinct schema “generations” within what was assumed to be one consistent format:

Years	Sheet Name	Key Columns	Notable Difference
2016–2018	national_dl	OCC_GROUP (uppercase, no separate industry/area columns)	No, I_GROUP or AREA columns exist at all-the file is implicitly national/cross-industry
2019	national_M2019_dl	o_group, i_group, area, occ_code, tot_emp, h_mean, a_mean (all lowercase)	Same conceptual columns as 2020+, but entirely lowercase-a transitional format
2020–2025	national_M20XX_dl	O_GROUP, I_GROUP, AREA, OCC_CODE, TOT_EMP, H_MEAN, A_MEAN (uppercase)	The format the pipeline was originally built against

Because the pipeline was originally designed and tested only against the 2020+ format, all three schema generations initially caused the same downstream symptom-fact_occupation_wages received 0 rows for 2016–2019-which required careful, layer-by-layer diagnosis rather than a single fix:

- **Diagnosis approach:** rather than assuming the reported symptom (a broken date lookup) was the actual fault, each transformation in the chain was checked in isolation using Data Preview, working backward from the Sink until the exact point where rows were being silently dropped was found.
- **Root cause 1 (case sensitivity):** the filter and join logic referenced fixed, uppercase column names; for 2019's all-lowercase file, every reference silently resolved to null.
- **Root cause 2 (missing columns):** the 2016–2018 files have no I_GROUP or AREA columns at all, since older national OES releases didn't need them.
- **Resolution:** a normalization Derived Column (normalizeSchema) was introduced immediately after the source, using ADF's byName() function (wrapped in toString(), as required for schema-drift-safe column access) with a tiered fallback: check the modern uppercase name, then the 2019 lowercase name, then fall back to a sensible default (e.g., 'cross-industry', '99') for columns that don't exist pre-2019. This lets a single, unchanged filter and join logic (filterValidRows) work correctly across all three file generations.

8.6 Decimal Type Consistency Through the Transformation Chain

Once the schema-drift issue was resolved, a second, unrelated problem surfaced: the native Unpivot transformation (used to convert the three wide OES metric columns-employment, hourly wage, annual wage-into a long, one-row-per-metric format matching the fact table's grain) repeatedly threw a type-mismatch error, even after every input column was explicitly and identically cast to decimal (18,4).

Despite extensive verification-confirming precision/scale at multiple checkpoints, restarting the Data Flow debug session, and clearing all cached state-the native Unpivot transformation continued to reject the columns as mismatched, pointing to a platform-level limitation rather than a logic error in the flow.

- **Resolution:** the native Unpivot transformation was replaced with an explicit, manually constructed equivalent: three parallel Derived Column branches (deriveEmp, deriveHMean, deriveAMean), each producing an identical two-column schema (metric_code, metric_value), followed by a Union transformation (unionMetrics, byName) to stack them into the required long format. This sidesteps Unpivot's internal type-validation logic entirely while producing an identical result and is now documented as the preferred pattern for this kind of reshape in ADF.

9. Limitations

- BLS OES ingestion is not fully automated-the absence of a public API for this specific national cross-industry cut of the data means new annual releases must be manually downloaded and staged before the (fully automated) transformation and load stages can run.
- The pipeline currently covers national-level aggregates only; state- or metro-level breakdowns are not modeled, which would substantially increase the size of dim_sector/dim_occupation and the corresponding fact tables.

- A known BLS data limitation is carried through as-is rather than resolved: series CES7000000002/3 report combined hours/earnings covering both the Professional Services and Leisure & Hospitality supersectors, meaning these two sectors cannot be distinguished on those two specific metrics.
- The PostgreSQL Flexible Server runs on a Burstable B1ms tier, appropriate for this portfolio's query volume but not representative of production-scale throughput or concurrency.
- Tableau dashboards are built on Extracts rather than a Live connection, meaning the visualized data reflects the most recent extract refresh rather than the database in real time-a deliberate trade-off for dashboard performance and to avoid keeping the database server running continuously.

10. Future Enhancements

A number of natural next steps were identified during development but kept out of scope for this iteration of the project, either because they add operational complexity beyond what a portfolio-scale workload needs, or because they depend on something outside the project's control (e.g., BLS API availability):

- **Automate OES ingestion:** if BLS ever exposes a public API for the national cross-industry cut of OES data (as it does for CES), replace the manual download/upload with a Web Activity-based pipeline matching the pattern already used for JOLTS and CES.
- **Incremental / watermark-based loads:** all three Data Flows currently reprocess a full parameter range (a series, a year range, or all 28 CES series) on every run via upsert. A watermark table tracking the last successfully loaded period per source would let future runs process only new or changed periods, reducing run time and computing cost as the historical window grows.
- **Pipeline monitoring and alerting:** add Azure Monitor alert rules or a Logic App notification on pipeline failure, rather than relying on manually checking the Monitor tab after each run.
- **Finer geographic granularity:** extend `dim_sector` and `dim_occupation` (and the corresponding fact tables) to support state- or metro-level breakdowns, which BLS publishes for both CES and OES, enabling regional labor market comparisons rather than national aggregates only.
- **CI/CD for ADF:** once the project's ARM/Data Flow exports are in source control, wire up an Azure DevOps or GitHub Actions pipeline to validate and auto-publish Data Factory changes on merge, rather than publishing manually from ADF Studio.
- **Automated data quality assertions:** formalize the manual verification queries from Section 7 into a small suite of automated checks (e.g., a Stored Procedure or Great Expectations-style validation step) that runs after each load and fails the pipeline if row counts or referential integrity checks don't match expectations.

11. Conclusion

This project delivers a complete, working analytics pipeline-from three heterogeneous public data sources, through a modular set of Azure Data Factory pipelines and Data Flows, into a properly normalized star schema in PostgreSQL, and finally into three purpose-built Tableau dashboards. Beyond the finished artifact, the process of building it surfaced and resolved a range of realistic, non-trivial data engineering problems-credential and timing issues, silent type mismatches, and multi-generational schema drift in a real government dataset-each documented here along with the reasoning used to diagnose and resolve them. Taken together, the project demonstrates practical competency across the full data engineering lifecycle: ingestion, transformation, modeling, and visualization, all on the Azure-native stack.